
Ayudantía de Preparación – Prueba N°2

Ayudante: Matías Fuentes

Esta ayudantía tiene como objetivo repasar los contenidos evaluados en la Prueba N°2: **estructuras iterativas** (while, do-while, for, simples o anidadas), **funciones** (prototipo, definición, llamada y traspaso de parámetros por valor), **ámbito de variables** (global y local), además de un repaso de la materia de la evaluación anterior (tipos de variables, sintaxis, if/else y switch).

1. ¿Qué ciclo utilizarías?

Un cajero automático simplificado debe solicitar al usuario que ingrese su PIN (un número entero de 4 dígitos). El sistema debe volver a solicitar el PIN **las veces que sea necesario** mientras el valor ingresado no tenga exactamente 4 dígitos (es decir, mientras sea menor a 1000 o mayor a 9999), ya que la primera solicitud siempre debe realizarse sin importar si el usuario ya conoce su PIN o no.

- a) Proponga y justifique brevemente **cuál de las tres estructuras iterativas** (while, do-while o for) es la más adecuada para este escenario, considerando cuántas veces como mínimo se debe ejecutar la solicitud del PIN.
- b) Implemente el programa completo en C utilizando la estructura que usted propuso, mostrando el mensaje "PIN valido" una vez que el usuario ingrese un valor correcto.

2. Encuentre y corrija los errores

El siguiente fragmento de código pretende calcular el promedio de N notas ingresadas por el usuario, y clasificar dicho promedio en 'Aprobado' (mayor o igual a 4.0) o 'Reprobado'. Sin embargo, el código contiene **4 errores** (de sintaxis, de lógica o de buenas prácticas) que impiden su correcto funcionamiento.

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     float nota, suma;
6     float promedio;
7
8     printf("Cuántas notas ingresara?: ");
9     scanf("%d", &n)
10
11     for (i = 1; i <= n; i++) {
12         printf("Ingrese nota %d: ", i);
13         scanf("%f", &nota);
14         suma = suma + nota;
15     }
16
17     promedio = suma / n;
18     printf("El promedio es: %.2f\n", promedio);
19
20     if (promedio >= 4.0)
21         printf("Aprobado\n");
22     else
23         printf("Reprobado\n")
24
25     return 0;
26 }
```

Identifique cada error señalando el número de línea, explique brevemente por qué es un error, y reescriba el código completo ya corregido.

3. Funciones en cadena

- a) Defina el prototipo y la implementación de una función llamada `esPrimo`, que reciba un número entero como parámetro y retorne un valor de tipo `int` que actúe como booleano (1 si el número es primo, 0 si no lo es). Considere que los números menores a 2 no son primos.
- b) Utilizando la función `esPrimo` definida en (a), implemente una segunda función llamada `contarPrimosEnRango`, que reciba dos números enteros `inicio` y `fin` como parámetros, y retorne (tipo `int`) la cantidad de números primos que existen en dicho rango (incluyendo ambos extremos). No es necesario escribir `main`.

Hint: Para verificar si un número es primo, basta con comprobar que no tenga divisores exactos entre 2 y el número menos 1.

4. ¿Por qué no funciona como se espera?

El siguiente programa fue diseñado para **duplicar** el valor de una variable ingresada por el usuario, utilizando una función llamada `duplicar`. Al compilarlo y ejecutarlo, el programa **no genera ningún error** y termina correctamente, pero el valor mostrado en pantalla **nunca cambia** respecto al valor ingresado originalmente.

```
1 #include <stdio.h>
2
3 void duplicar(int numero) {
4     numero = numero * 2;
5 }
6
7 int main() {
8     int valor;
9
10    printf("Ingrese un numero: ");
11    scanf("%d", &valor);
12
13    duplicar(valor);
14
15    printf("El valor duplicado es: %d\n", valor);
16
17    return 0;
18 }
```

- Explique, en sus propias palabras, **por qué** el programa no produce el resultado esperado, haciendo referencia explícita a la forma en que C traspasa los parámetros a las funciones.
- Proponga una **corrección** al código (puede modificar la función, la forma en que se invoca, o ambas) de manera que el valor mostrado en pantalla sí corresponda al valor original duplicado.

5. Implemente el código solicitado

Escriba un programa en C que, utilizando **ciclos anidados**, imprima en pantalla un patrón triangular de asteriscos como el siguiente, para un valor de N ingresado por el usuario (en el ejemplo, $N = 5$):

```
*
* *
* * *
* * * *
* * * * *
```

Adicionalmente, el programa debe declarar una variable **global** llamada `totalAsteriscos` que vaya acumulando la cantidad total de asteriscos impresos a lo largo de todo el patrón, y mostrarla en pantalla una vez finalizado el dibujo.

Hint: Recuerde que una variable declarada fuera de cualquier función tiene ámbito **global**, por lo que puede ser modificada directamente desde dentro de un ciclo en `main` sin necesidad de pasarla como parámetro.

6. Cacería de Fantasmas

Un equipo de *Ghostbusters* utiliza un detector que capta señales enteras positivas de hasta 4 dígitos, leídas de forma continua hasta que se ingrese una señal **inválida** (negativa o igual a 0). Si una señal es **palíndromo**, se **captó** un fantasma; si además su **suma de dígitos es divisible por 3**, el fantasma queda **atrapado**.

Al finalizar, el programa debe mostrar la cantidad de fantasmas captados y atrapados, el **porcentaje** de atrapados respecto a los captados (evitando división por cero), y un mensaje final: "Ghostbuster profesional!" si atrapó 5 o más, "Probablemente tu eres el fantasma..." si no atrapó ninguno, o "Sigue practicando tu caceria." en cualquier otro caso.

Hint: Reutilice la lógica de invertir un número y sumar sus dígitos con % y /. La condición de .atrapado" solo se evalúa si el número ya es palíndromo.

7. Cirugía de Dígitos

Escriba un programa en C que solicite al usuario un número entero positivo de exactamente **3 dígitos**, y luego solicite un **dígito de reemplazo** (entre 0 y 9) junto con la **posición** que se desea modificar (1 para las unidades, 2 para las decenas, 3 para las centenas).

El programa debe:

- a) **Desarmar** el número original, dígito a dígito, utilizando el operador módulo (%) y la división entera (/), guardando cada dígito (unidades, decenas, centenas) en variables separadas.
- b) **Reemplazar** el dígito correspondiente a la posición indicada por el nuevo dígito ingresado.
- c) **Rearmar** el número final a partir de los tres dígitos (ya con el reemplazo aplicado), multiplicando cada uno por la potencia de 10 que corresponda según su posición.

Por ejemplo, si el número ingresado es 123, el dígito de reemplazo es 4 y la posición es 2 (decenas), el resultado debe ser 143.

Hint: Para desarmar el número: `unidades = numero % 10`, luego `numero = numero / 10` para obtener decenas = `numero % 10`, y así sucesivamente. Para rearmar: `numeroFinal = unidades + decenas * 10 + centenas * 100`.

Soluciones

Solución Ejercicio 1: ¿Qué ciclo utilizarías?

a) Justificación: La estructura más adecuada es `do-while`. Esto se debe a que el enunciado indica explícitamente que **la primera solicitud del PIN siempre debe realizarse**, sin importar si el valor sería válido o no; es decir, el bloque de instrucciones debe ejecutarse **al menos una vez** antes de evaluar la condición. Tanto `while` como `for` evalúan la condición **antes** de ejecutar las instrucciones, por lo que si se usaran, sería necesario duplicar código o forzar una primera lectura fuera del ciclo; `do-while` evita ese problema evaluando la condición al final.

b) Implementación:

```
1 #include <stdio.h>
2
3 int main() {
4     int pin;
5
6     do {
7         printf("Ingrese su PIN (4 digitos): ");
8         scanf("%d", &pin);
9
10        if (pin < 1000 || pin > 9999) {
11            printf("PIN invalido, intente nuevamente.\n");
12        }
13    } while (pin < 1000 || pin > 9999);
14
15    printf("PIN valido\n");
16
17    return 0;
18 }
```

Explicación: Al usar `do-while`, el bloque `printf/scanf` se ejecuta primero sin condiciones, y solo después se evalúa si el PIN ingresado cumple con el rango de 4 dígitos, repitiendo la solicitud tantas veces como sea necesario.

Solución Ejercicio 2: Encuentre y corrija los errores

- **Línea 9:** Falta el punto y coma (;) al final de la instrucción `scanf("%d", &n)`. Toda sentencia en C debe terminar con punto y coma.
- **Línea 11:** La variable `i` se utiliza en el `for` sin haber sido declarada previamente. Debe declararse como `int i`; junto con las demás variables enteras.
- **Línea 6:** La variable `suma` no fue inicializada en 0 antes de comenzar a acumular valores dentro del ciclo. Al contener un valor "basura" en memoria, el promedio calculado sería incorrecto.
- **Línea 22:** Falta el punto y coma (;) al final de la instrucción `printf("Reprobado\n")` dentro del `else`.

Código corregido:

```
1 #include <stdio.h>
2
3 int main() {
4     int n, i;
5     float nota, suma;
6     float promedio;
7
8     suma = 0;
9
10    printf("Cuantas notas ingresara?: ");
11    scanf("%d", &n);
12
13    for (i = 1; i <= n; i++) {
14        printf("Ingrese nota %d: ", i);
15        scanf("%f", &nota);
16        suma = suma + nota;
17    }
18
19    promedio = suma / n;
20    printf("El promedio es: %.2f\n", promedio);
21
22    if (promedio >= 4.0)
23        printf("Aprobado\n");
24    else
25        printf("Reprobado\n");
26
27    return 0;
28 }
```

Explicación: Este ejercicio combina errores típicos de un programador principiante: olvidar puntos y coma, usar variables sin declarar, y no inicializar acumuladores antes de un ciclo. Este último es especialmente importante, ya que el programa puede **compilar sin problemas** y aun así entregar resultados incorrectos.

Solución Ejercicio 3: Funciones en cadena

```
1 // Prototipos
2 int esPrimo(int numero);
3 int contarPrimosEnRango(int inicio, int fin);
4
5 // a) Determina si un numero es primo
6 int esPrimo(int numero) {
7     int i;
8
9     if (numero < 2) {
10         return 0;
11     }
12
13     for (i = 2; i < numero; i++) {
14         if (numero % i == 0) {
15             return 0; // Encontro un divisor exacto, no es primo
16         }
17     }
18
19     return 1; // No encontro divisores, es primo
20 }
21
22 // b) Cuenta cuantos primos hay en un rango, reutilizando esPrimo
23 int contarPrimosEnRango(int inicio, int fin) {
24     int contador = 0;
25     int numeroActual;
26
27     for (numeroActual = inicio; numeroActual <= fin; numeroActual++) {
28         if (esPrimo(numeroActual) == 1) {
29             contador++;
30         }
31     }
32
33     return contador;
34 }
```

Explicación: La función `esPrimo` encapsula la lógica de verificación individual, retornando 1 o 0 como si fuera un booleano. La función `contarPrimosEnRango` **reutiliza** esta lógica llamando a `esPrimo` dentro de un ciclo `for`, ejemplificando cómo una función puede apoyarse en otra ya definida sin repetir código (principio de modularidad).

Solución Ejercicio 4: ¿Por qué no funciona como se espera?

a) Explicación: En C, los parámetros se traspasan a las funciones **por valor**. Esto significa que, al llamar a `duplicar(valor)`, la función recibe una **copia** del contenido de la variable `valor`, la cual se almacena en el parámetro local `numero`. Cualquier modificación realizada sobre `numero` dentro de la función afecta únicamente a esa copia local, y **no** a la variable original `valor` declarada en `main`. Por eso, al salir de la función, `valor` conserva su valor original sin cambios.

b) Corrección propuesta: Una forma sencilla de solucionar esto (sin usar punteros) es hacer que la función **retorne** el valor duplicado, y luego asignar ese resultado a la variable en `main`:

```
1 #include <stdio.h>
2
3 int duplicar(int numero) {
4     numero = numero * 2;
5     return numero;
6 }
7
8 int main() {
9     int valor;
10
11     printf("Ingrese un numero: ");
12     scanf("%d", &valor);
13
14     valor = duplicar(valor);
15
16     printf("El valor duplicado es: %d\n", valor);
17
18     return 0;
19 }
```

Explicación: Al cambiar el tipo de retorno de la función de `void` a `int` y hacer que devuelva el resultado del cálculo, podemos capturar ese valor en `main` mediante la asignación `valor = duplicar(valor)`, logrando así el efecto deseado sin necesidad de modificar la variable original directamente dentro de la función.

Solución Ejercicio 5: Implemente el código solicitado

```
1 #include <stdio.h>
2
3 // Variable global: accesible desde cualquier funcion del programa
4 int totalAsteriscos = 0;
5
6 int main() {
7     int n, fila, columna;
8
9     printf("Ingrese el valor de N: ");
10    scanf("%d", &n);
11
12    for (fila = 1; fila <= n; fila++) {
13        for (columna = 1; columna <= fila; columna++) {
14            printf("* ");
15            totalAsteriscos++; // Modificamos directamente la variable global
16        }
17        printf("\n");
18    }
19
20    printf("Total de asteriscos impresos: %d\n", totalAsteriscos);
21
22    return 0;
23 }
```

Explicación: El ciclo externo (fila) controla cuántas filas se dibujan, mientras que el ciclo interno (columna) controla cuántos asteriscos se imprimen en cada fila, aumentando en 1 en cada iteración externa. Como totalAsteriscos se declaró **fuera** de cualquier función (ámbito global), puede ser incrementada directamente desde dentro del ciclo anidado en main sin necesidad de traspasarla como parámetro ni retornarla.

Solución Ejercicio 6: Cacería de Fantasmas

```
1 #include <stdio.h>
2
3 int main() {
4     int senal, original, invertido, resto, sumaDigitos, temp;
5     int captados = 0, atrapados = 0;
6     float porcentaje;
7
8     printf("Ingrese una senal (negativo o 0 para terminar): ");
9     scanf("%d", &senal);
10
11     while (senal > 0) {
12         original = senal;
13
14         // Invertimos el numero para verificar si es palindromo
15         invertido = 0;
16         temp = senal;
17         while (temp > 0) {
18             resto = temp % 10;
19             invertido = invertido * 10 + resto;
20             temp = temp / 10;
21         }
22
23         if (original == invertido) {
24             captados++;
25
26             // Sumamos los digitos del numero
27             sumaDigitos = 0;
28             temp = original;
29             while (temp > 0) {
30                 sumaDigitos = sumaDigitos + (temp % 10);
31                 temp = temp / 10;
32             }
33
34             if (sumaDigitos % 3 == 0) {
35                 atrapados++;
36             }
37         }
38
39         printf("Ingrese la siguiente senal (negativo o 0 para terminar): ");
40         scanf("%d", &senal);
41     }
42
43     printf("Fantasmas captados: %d\n", captados);
44     printf("Fantasmas atrapados: %d\n", atrapados);
45
46     if (captados > 0) {
47         porcentaje = (float) atrapados / captados * 100;
48         printf("Porcentaje atrapados: %.2f%%\n", porcentaje);
49     } else {
50         printf("No se detectaron fantasmas.\n");
51     }
52
53     if (atrapados >= 5) {
54         printf("Ghostbuster profesional!\n");
55     }
```

```
55     } else if (atrapados == 0) {
56         printf("Probablemente tu eres el fantasma...\n");
57     } else {
58         printf("Sigue practicando tu caceria.\n");
59     }
60
61     return 0;
62 }
```

Explicación: Se reutiliza la técnica de invertir un número con % y / para detectar palíndromos. La condición de 'atrapado' (`sumaDigitos % 3 == 0`) solo se evalúa **dentro** del bloque que ya confirmó el palíndromo, por lo que un fantasma atrapado siempre fue antes captado. Para el porcentaje se castea a `float` antes de dividir, y se valida `captados > 0` para evitar división por cero.

Solución Ejercicio 7: Cirugía de Dígitos

```
1 #include <stdio.h>
2
3 int main() {
4     int numero, unidades, decenas, centenas;
5     int nuevoDigito, posicion, numeroFinal;
6
7     printf("Ingrese un numero de 3 digitos: ");
8     scanf("%d", &numero);
9     printf("Ingrese el digito de reemplazo (0-9): ");
10    scanf("%d", &nuevoDigito);
11    printf("Ingrese la posicion a reemplazar (1=unidad, 2=decena, 3=centena): "
12           );
13    scanf("%d", &posicion);
14
15    // a) Desarmamos el numero digito a digito
16    unidades = numero % 10;
17    numero = numero / 10;
18    decenas = numero % 10;
19    numero = numero / 10;
20    centenas = numero % 10;
21
22    // b) Reemplazamos el digito en la posicion indicada
23    if (posicion == 1) {
24        unidades = nuevoDigito;
25    } else if (posicion == 2) {
26        decenas = nuevoDigito;
27    } else if (posicion == 3) {
28        centenas = nuevoDigito;
29    }
30
31    // c) Rearmamos el numero final
32    numeroFinal = unidades + decenas * 10 + centenas * 100;
33
34    printf("El numero resultante es: %d\n", numeroFinal);
35
36    return 0;
37 }
```

Explicación: El número se desarma extrayendo cada dígito con % 10 (resto de la división) y reduciéndolo con / 10 (división entera), igual que en ejercicios anteriores. Luego, según la `posicion` indicada, se reemplaza solo la variable correspondiente, y finalmente se rearma el número multiplicando cada dígito por la potencia de 10 asociada a su posición (unidades $\times 1$, decenas $\times 10$, centenas $\times 100$) y sumando los resultados.